



浙江大学
ZHEJIANG UNIVERSITY

HPC101 大作业实验报告

CONV / ZGEMM / TRSM 三项赛题优化

姓名 曹 绚

学号 3240101033

姓名 胡尊昊

学号 3250103895

姓名 王铭翔

学号 3250102670

2026 年 9 月 5 日

摘 要

本报告总结 HPC101 大作业（鲲鹏高性能计算全球挑战赛 S2 赛季）三道赛题的优化工作：二维卷积 CONV、复数矩阵乘 ZGEMM 与下三角求解 TRSM。三项优化均在鲲鹏 920 系列 aarch64 集群上完成，并遵循三条共同原则：数值结果必须满足官方误差阈值；线程与 NUMA 绑定必须与集群的实际 CPU 配给方式匹配；一切性能判断以集群实测为准。

CONV 采用 OpenMP 输出行并行与 ARM NEON 向量化，并复用相邻四行的输入数据，四组用例全部 PASS 且最大误差为零，性能达到 454.98 至 534.14 GFLOPS，较官方基线提升约 13.4 至 14.0 倍。ZGEMM 从 NEON FCMLA 基线出发，构建 SVE 512-bit 的 MR12 微内核，配合全进程 NUMA 绑定与收窄的列块参数 $NC=192$ ，三用例合计达到 3125.28 GFLOPS，并对 Strassen 算法路线做了完整的可行性验证与否决。TRSM 面向三个形状差异极大的用例构建递归与分块混合求解路径，以 W-technique 消除对角块求解的串行链、以二维切分降低宽矩阵更新的面板重复读取，综合吞吐量达到 1182.1 GFLOPS，三组用例全部满足 10^{-12} 误差阈值。

关键词：高性能计算；鲲鹏 920；OpenMP；NEON/SVE；数值线性代数；性能优化

目录

1 大作业概述	4
1.1 题目来源与评测方式	4
1.2 实验环境	4
1.3 成员与分工	4
1.4 贯穿三题的优化原则	4
2 题目一：CONV 二维卷积优化	5
2.1 题目与计算模型	5
2.2 基线分析与优化思路	6
2.3 优化实现	6
2.4 正确性与性能结果	7
2.5 尝试、取舍与局限	8
3 题目二：ZGEMM 复数矩阵乘优化	8
3.1 题目与计算模型	8
3.2 基线分析与优化思路	8
3.3 优化实现	9
3.4 正确性与性能结果	11
3.5 尝试、取舍与局限：发射率上限与 Strassen 否决	11
4 题目三：TRSM 下三角求解优化	12
4.1 题目与计算模型	12
4.2 基线分析与优化思路	12
4.3 优化实现	13
4.4 正确性与性能结果	15
4.5 尝试、取舍与局限	16
5 总结与反思	16
5.1 三题最终结果汇总	16
5.2 共性经验与反思	16

1 大作业概述

1.1 题目来源与评测方式

本课程大作业对应鲲鹏高性能计算全球挑战赛（S2 赛季）的三道赛题 [1]：CONV 二维卷积、ZGEMM 复数矩阵乘与 TRSM 下三角求解。三道题共用同一套评测框架：官方提供 bench 程序与固定规模的测试用例，程序输出运行时间、GFLOPS、最大误差与校验结果，其中只有误差低于阈值且显示 PASS 的成绩才有效。三道题的误差阈值分别为 CONV 的 10^{-5} 、ZGEMM 的 10^{-10} （绝对误差）与 TRSM 的 10^{-12} ，评分主要依据 GFLOPS 性能。

1.2 实验环境

题目	平台	编译器	并行与绑定	数学库
CONV	浙大鲲鹏 920 集群 login04	GCC 10.3.1	OpenMP 38 线程, numactl -N 1	无
ZGEMM	深圳超算鲲鹏 920	GCC 12.3.1 (HPCKit)	OpenMP 38 线程, close 绑定 + taskset node0	无（自研微内核）
TRSM	深圳超算鲲鹏 920F	GCC + BiSheng HPCKit	OpenMP 38 线程, numactl -N 1	KBLAS v25.2.1（单线程）

表 1: 三题的实验环境

三题的硬件细节如表 1 所示。浙大集群的鲲鹏 920 节点为 128 个逻辑核、4 个 NUMA 节点，仅支持 NEON (asimd) 指令集；深圳超算的鲲鹏 920 节点主频 2.0 GHz，单节点 608 核、32 个 NUMA 节点（每节点 38 核），支持 SVE 且向量长度为 512 bit，TRSM 所用的 920F 节点进一步支持 SVE2 与 SME。三题的评测环境都按单 NUMA 节点 38 个线程设计，其中 ZGEMM 的批处理系统 CPU 配给方式不定，需要提交脚本自行探测。

1.3 成员与分工

三道赛题由小组三人分别负责，分工如表 2 所示。各题章节由对应负责人撰写，实验环境、优化原则与总结部分由三人共同讨论定稿。

1.4 贯穿三题的优化原则

三项工作分属不同的计算结构，但优化过程中保留了三条相同的原则。

题目	负责人	主要工作
CONV	胡尊昊	输出行并行、NEON 向量化与跨输出行数据复用
ZGEMM	王铭翔	SVE 微内核、NUMA 绑定纪律与分块参数调优
TRSM	曹绚	形状混合递归/分块算法、W-technique 与 KBLAS 集成

表 2: 成员分工

第一，数值一致性优先。任何改变浮点运算顺序的优化都必须对照官方校验结果：CONV 通过保持卷积核行、列的累加顺序与融合乘加语义做到误差为零；ZGEMM 与 TRSM 的最终误差均比阈值低四个数量级以上。第二，绑定纪律。OpenMP 线程与内存页面必须固定在单个 NUMA 节点上，否则线程在节点间迁移会使微内核的缓存局部性全部失效，这一点在 ZGEMM 中直接表现为约 35 倍的性能差距。第三，以实测为准。共享集群上的性能受瞬时负载与节点体质影响，纸面分析与本地模拟都只用于提出假设，最终结论全部以集群同批对照为准，负结果与正收益同样记录在案。

为便于横向对照，后续三道题目的章节采用统一结构：题目与计算模型、基线分析与优化思路、优化实现、正确性与性能结果、尝试取舍与局限。

2 题目一：CONV 二维卷积优化

2.1 题目与计算模型

本题要求优化二维有效卷积。输入为二维浮点矩阵 I 与卷积核 K ，输出不做填充，尺寸为

$$H_o = H_i - H_k + 1, \quad W_o = W_i - W_k + 1, \quad (1)$$

每个输出元素的计算形式为

$$O_{j,i} = \sum_{r=0}^{H_k-1} \sum_{c=0}^{W_k-1} I_{j+r, i+c} K_{r,c}, \quad (2)$$

官方测试程序先运行参考实现，再运行优化后的 `conv2d.c` 并逐元素比较，要求最大误差不超过 10^{-5} 。四组正式测试规模如表 3 所示，卷积核从 39×39 到 81×81 不等，输出矩阵的行列数均较大。

测试编号	输入矩阵	卷积核	重复次数
1	4096×6144	39×39	1
2	6144×4096	41×41	1
3	4256×6390	55×55	1
4	6390×4256	81×81	1

表 3: CONV 官方测试用例

2.2 基线分析与优化思路

官方原始实现采用三层循环：最外层遍历输出行，中间层遍历输出列，内层遍历卷积核的行与列，每个输出元素独立完成全部乘加累积，结构如下：

```
for each output row j:
  for each output column i:
    sum = 0
    for each kernel row r:
      for each kernel column c:
        sum += input[j + r][i + c] * kernel[r][c]
    output[j][i] = sum
```

这种写法逻辑清晰，但存在三个主要问题。第一，输出元素之间没有显式的线程并行，基线仅依赖编译器自动向量化，四组用例只有约 33.4 至 39.7 GFLOPS。第二，标量循环没有充分使用鲲鹏处理器的 NEON 向量单元。第三，相邻输出行使用了大量相同的输入行，但原始循环没有在多个输出行之间复用这些数据。

此外，官方校验对单精度浮点结果逐元素比较。若简单改变累加顺序，虽然数学上等价，但浮点舍入会发生变化，可能导致误差增大。因此优化必须同时考虑计算速度与累加顺序。

据此确定的优化思路对应三个性能问题逐项展开：以 OpenMP 输出行并行补齐线程级并行，以 NEON 向量化提高单指令计算吞吐量，以相邻四行的数据复用减少重复的输入读取；数值一致性则作为约束贯穿全部实现细节。

2.3 优化实现

2.3.1 OpenMP 输出行并行

输出矩阵的不同行写入互不重叠的内存区域，不存在写冲突。程序使用 OpenMP 将输出行静态分配给线程，最终版本以四行为一组进行调度，每个线程负责若干组相邻输出行；输出高度不能被 4 整除时保留两行与单行收尾路径。schedule(static) 使任

务分配保持稳定，每个线程连续处理一段输入与输出区域，与 NUMA 绑定下的局部性要求一致。

2.3.2 ARM NEON 向量化

鲲鹏 920 的 NEON 单元可在一条指令中处理 4 个单精度浮点数。程序将连续的输出列组织为 4 元素向量，用 `vld1q_f32` 读取输入、向量乘加完成累积、`vst1q_f32` 写回。每个输出行按 8 列小块组织并使用两个向量累加器，四行合并后共有 8 个向量累加器，用于隐藏向量乘加指令的等待时间，避免单一累加链造成的流水线停顿。

2.3.3 相邻四行的数据复用

相邻输出行的卷积窗口在输入方向上高度重叠：计算输出行 j 至 $j + 3$ 只需要输入行 j 至 $j + H_k + 2$ 。程序按输入行自上而下遍历，在同一次输入行读取之后，分别使用不同的卷积核行系数更新四个输出行的累加器，相比分别计算四个输出行显著减少了重复的输入向量读取。最终选择四行、八列的向量小块，是因为它在数据复用与寄存器占用之间取得了较好的平衡。

2.3.4 保持数值一致性

向量路径中每个输出通道仍按卷积核行、卷积核列的顺序累加，编译器在鲲鹏平台上将向量乘加识别为融合乘加指令，标量收尾路径显式使用 `fmaf`，从而与参考实现保持一致的单精度融合乘加语义。对于输出宽度不能被 8 整除的情况，依次使用四列向量尾部与最多三列标量尾部；高度不能被 4 整除的情况使用两行或单行尾部。这样既覆盖任意合法尺寸，也不会访问输入矩阵边界之外的内存。

2.4 正确性与性能结果

在四组正式数据上，优化版本的最大误差均为 $0.00e + 00$ ，全部显示 PASS；非整除宽度与高度的小尺寸边界测试同样全部通过。正式结果如表 4 所示，与官方基线的对比如表 5 所示。

测试	输入与卷积核	时间 / ms	GFLOPS	最大误差	校验
1	$4096 \times 6144, 39 \times 39$	151.70	496.8654	0.00e+00	PASS
2	$6144 \times 4096, 41 \times 41$	182.94	454.9808	0.00e+00	PASS
3	$4256 \times 6390, 55 \times 55$	311.45	517.1831	0.00e+00	PASS
4	$6390 \times 4256, 81 \times 81$	647.35	534.1367	0.00e+00	PASS

表 4: CONV 优化版本正式测试结果

测试	基线 GFLOPS	优化 GFLOPS	基线时间 / ms	优化时间 / ms	加速比
1	35.4016	496.8654	2129.15	151.70	14.035 倍
2	33.4428	454.9808	2488.90	182.94	13.603 倍
3	37.4192	517.1831	4304.59	311.45	13.821 倍
4	39.7441	534.1367	8699.96	647.35	13.443 倍

表 5: CONV 与官方原始实现的对比

四组加速比均超过 13 倍，最高性能出现在第四组，达到 534.14 GFLOPS；第二组相对较低，主要与输入矩阵布局、卷积核尺寸以及节点运行时的缓存与内存状态有关。需要说明的是，性能测试会受到共享集群瞬时负载与处理器频率变化的影响，表中数据是在同一节点、相同线程数与 NUMA 绑定条件下取得的正式记录；正确性结果不受性能波动影响。

2.5 尝试、取舍与局限

优化过程中比较了三类输出行分组方式：单行版本实现最简单但输入行重复读取最多；两行版本有明显提升；四行版本在平台上取得最好综合结果。八行版本虽然理论上可以进一步复用输入行，但需要更多条件判断与卷积核系数更新，四组实测性能均低于四行版本，因此被撤回。此外，在函数参数中加入指针别名限制没有带来稳定收益，第二组测试还出现回落，最终没有采用。这说明针对特定处理器的优化不能只依据理论推测判断，必须在官方数据与固定运行条件下反复验证。

3 题目二：ZGEMM 复数矩阵乘优化

3.1 题目与计算模型

本题要求优化双精度复数矩阵乘

$$C = \alpha AB + \beta C, \quad A \in \mathbb{C}^{m \times k}, B \in \mathbb{C}^{k \times n}, C \in \mathbb{C}^{m \times n}, \quad (3)$$

官方 bench 以 $8mnk$ 计量 FLOPs，输出运行时间、GFLOPS、最大绝对误差与校验结果，误差阈值为 10^{-10} 。三组正式用例如表 6 所示，特征是两个大 n 方阵、 k 只有 2 个 KC 面板，以及一个细长形状、 k 为 4 个 KC 面板的用例。

3.2 基线分析与优化思路

起点是移植到深圳超算的 NEON FCMLA 版本（MR6、NR2、KC128、MC384、NC512），三用例合计约 1110 GFLOPS。该版本在浙大 920B 节点上每用例约 900

用例	规模 $m \times n \times k$	形状特征
Case 1	$7427 \times 7427 \times 256$	大 n , k 仅 2 个 KC 面板
Case 2	$14848 \times 14848 \times 256$	最大 n , k 仅 2 个 KC 面板
Case 3	$37360 \times 8192 \times 512$	细长 m , k 为 4 个 KC 面板

表 6: ZGEMM 三组正式测试规模

GFLOPS, 移植到深圳节点后有两处瓶颈: NEON 每条指令只处理 2 个复数, 没有用上深圳节点 512 bit 的 SVE 单元; 官方用例的 k 只有 256 或 512 (2 至 4 个 KC 面板), $NC=512$ 使 tile 数过少, B 打包开销占比高, 38 线程的尾部不均衡。

优化路线分四步: 修正线程与 NUMA 绑定, 微内核升级为 SVE MR12, 收窄 NC 并按 NUMA 行分区重排任务, 最后用微基准与算法对照验证剩余空间。版本演进如表 7 所示, 最大的提升 (约 35 倍) 来自绑定修正, 内核与分块参数的改进贡献了另外约 2.8 倍 (1110 到 3125)。

版本	关键改动	三用例合计 / GFLOPS	说明
FCMLA 基线	NEON, MR6 NR2 KC128	~1110	深圳超算移植版
SVE MR6	LD1RO 寄存器内广播	1222	A 载入 1.33→0.58 B/flop
SVE MR12	微内核加宽至 12 行	~2077	B 载入摊薄至 0.42 B/flop
NUMA v4 + NC=192	行分区 + 窄列块	3081.9	run.sh v7 判题机模拟
最终复测	同源环境三方 A/B	3125.28	2026-09-04, 节点 cn22993

表 7: ZGEMM 版本演进与三用例合计性能

3.3 优化实现

3.3.1 线程绑定与 NUMA 纪律

绑核纪律对性能起决定作用。批处理系统默认把 38 个配额 CPU 打散到各 NUMA 节点, `numactl -N 1` 又把线程全绑到没有配额核的节点上, 首轮验收只有约 10 GFLOPS。即使整节点独占、CPU 全部空闲, 不绑核的 38 线程也只有约 10 GFLOPS; `OMP_PROC_BIND=close` 与 `OMP_PLACES=cores` 一次修复带来约 35 倍提升。原因是 FCMLA 微内核的 A/B 微面板随线程迁移在核间搬动, L1/L2 缓存全部失效。

内存页面是第二处问题。bench 的矩阵初始化是 OpenMP 并行的 first-touch 分配，页面落在哪个节点由进程允许的 CPU 集决定：进程可见全节点时页面散到 32 个 NUMA 节点，38 个计算线程大部分访问远端内存，实测约慢 2 倍。修复方式是把整个进程（含初始化）用 taskset 限制在 node0 的 38 个 CPU 内，使全部矩阵页面与计算线程同节点；numactl --interleave=all 反而更慢，说明正确策略是全压 node0。这些处理写入 run.sh：线程数从绑定后的进程读取而非 nproc，并清除 HPCKit 模块导出的 GOMP_CPU_AFFINITY，避免覆盖 OpenMP 绑定。

3.3.2 SVE 微内核：MR12 与 LD1RO

深圳超算的鲲鹏 920 支持 SVE，向量长度 512 bit，即每条 svcmla 指令处理 4 个复数。官方编译命令固定为 gcc -O3 且不能加 -march，因此 SVE 通过源码内的 #pragma GCC target("arch=armv8.2-a+sve") 打开，实测与命令行开启等价。

微内核分两步演进。第一步改 A 数据通路：pack A 不再预展开成广播（4 倍内存），每个元素紧凑存储 1 个复数，微内核用 svld1rq_f64（LD1RO）一次加载并复制满整个向量，A 载入量从 1.33 降到 0.58 B/flop，三用例合计从约 1110 提升到 1222 GFLOPS。第二步把微内核从 MR6 加宽到 MR12：12 行共用两个 B 向量载入，B 载入摊薄到 0.42 B/flop，每个 k 步的计算密度翻倍、epilogue 减半。加宽的前提是寄存器预算：24 个累加器（z0-z23）、2 个 B 向量（z24、z25）与 3 个轮换的 LD1RO 暂存（z26-z28）共 29 个 z 寄存器，未超过 32 个的限额；代价是 24 个累加器必须写成命名寄存器变量，因为 SVE 类型不能做数组元素。该版本三用例合计约 2077 GFLOPS，接近 FCMLA 基线的 2 倍。

3.3.3 NUMA 行分区与 NC 收窄

此前“线程散到多节点必输”的结论在分区方向上被修正：问题不在跨节点本身，而在列分区与 first-touch 页面分布错位。NUMA v4 改为按行（ic）分区，使 C 的写入、A 的打包与 bench 的 first-touch 页面同构，并用跨节点窃取补齐小 n 时的 tile 数；Case 3 在 304 线程下从 696 提升到 1086.9 GFLOPS。判题环境按 38 核设计，最终成绩由单节点路径决定，行分区保留在提交源码中作为多节点路径。

决定最终成绩的是 NC 参数。官方用例的 k 只有 2 至 4 个 KC 面板，NC=512 时每个 tile 的 B 打包工作量大而 tile 数少（Case 1 仅 $3 \times 15 = 45$ 个 tile，38 线程分不均）。38 线程单节点的 NC 扫描结果如表 8 所示：NC 收窄到 192 后 tile 数增至约 2.7 倍，尾部不均衡被摊平，192 列的打包 B（384 KB）在 L2 中复用更好，合计从约 2077 提升到约 3073 GFLOPS。

NC=192 是平台区高点，NC=192 至 256 的成绩都在 3000 的 2% 以内。官方编译命令不支持 -D，参数只能写进源码，最终把 NC=192 设为默认值，形成提交版 zgemm_nc192.c。

NC	Case 1	Case 2	Case 3	合计
512 (旧默认)	675	691	711	~2077
256	968–980	1033–1037	1000–1004	~3015
192	980–1003	1052–1054	1021–1029	~3073
128	860–957	993–1010	1026–1029	~2979

表 8: 38 线程单节点 NC 扫描 (3 runs/case)

3.4 正确性与性能结果

提交版 run.sh v7 在整节点独占下按判题流程模拟, 得到 981.8 / 1058.9 / 1041.2, 合计 3081.9 GFLOPS; 随后在判题机同源环境下三方 A/B 复测, 成绩如表 9 所示, 合计 3125.28 GFLOPS, 与模拟结果相差 +1.4%, 在节点方差范围内。

用例	规模 $m \times n \times k$	GFLOPS	最大误差	校验
Case 1	$7427 \times 7427 \times 256$	1009.93	2.57×10^{-12}	PASS
Case 2	$14848 \times 14848 \times 256$	1071.82	2.70×10^{-12}	PASS
Case 3	$37360 \times 8192 \times 512$	1043.53	4.27×10^{-12}	PASS

表 9: ZGEMM 最终判题机同源复测结果 (38 线程, 节点 cn22993)

正确性方面, 除三组官方用例外, $37 \times 41 \times 53$ 、128 立方、 $1 \times 1 \times 1$ 、 $511 \times 7 \times 3$ 等奇尺寸回归全部 PASS, 误差稳定在 10^{-12} 量级, 相对 10^{-10} 阈值有约两个数量级裕度。提交包由三部分组成: zgemm.c 为 FCMLA 回退源, zgemm_nc192.c 为 SVE 主路径, run.sh 内含 SVE 检测、 $8 \times 8 \times 8$ 冒烟测试 (非 SVE 硬件回退 FCMLA) 与绑定逻辑。

3.5 尝试、取舍与局限: 发射率上限与 Strassen 否决

为确定剩余优化空间, 实验直接测量硬件上限: 对纯计算循环做防死代码消除处理后, svcmla 发射率为 24.8 GFLOPS/核 ($2.0 \text{ GHz} \times 12.4 \text{ flop/cycle}$), 38 核上限为 942 GFLOPS/用例。三用例实测 970 至 1050 GFLOPS 略超该值, 原因是节点级 L2/L3 共享下 B 面板被多核复用, 缓存命中使整节点聚合率高于单核 L1 常驻率。常规 GEMM 的每用例成绩已达到发射率上限的 103% 至 111%, 继续提高只能减少乘法次数, 即 Strassen 类算法。

两级 Strassen 把乘法量降到 49/64, 但前提苛刻: 只有子问题 GEMM 速率保持在全尺寸的 $q \geq 0.994$ 才能达到 4000 GFLOPS 的假设目标。为此实现了完整的两级版本 (本地 150 个回归形状加 4 个两级大形状全部 PASS, 误差约 10^{-12}), 并在集群上与 v7 同源对照。结果否定了该路线: str1 合计 511.90 GFLOPS (慢 6.1 倍), str2 合计 136.30

GFLOPS（慢 22.9 倍），正确性全部通过但性能很差。原因是子问题速率明显下降：1 级 Strassen 的子问题（半尺寸、 $k=128$ ）只有全尺寸的 77%-96%，2 级（四分之一尺寸、 $k=64$ ）只有 25%-88%，实测 $q = 0.25-0.79$ ，远低于 0.994 的要求。这不是实现问题，而是官方形状决定的： k 只有 256/512，拆两层后 $k = 64$ 只剩半个 KC 面板，子问题缩小后 L3 复用收益也消失。本地 x86 模拟曾预测 str2 约为 v7 的 1.24 倍，与集群实测的 0.13 倍差距很大，说明性能判断必须以集群实测为准。

综上，38 线程单节点下 3125 GFLOPS 已接近该硬件上限，常规 GEMM 参数进入平台区，Strassen 路线被实测否定，提交版维持 v7 包。

4 题目三：TRSM 下三角求解优化

4.1 题目与计算模型

给定下三角矩阵 L 与右端矩阵 B ，程序求解

$$LX = B, \quad L \in \mathbb{R}^{m \times m}, \quad X, B \in \mathbb{R}^{m \times n}. \quad (4)$$

本题的 L 为下三角矩阵，求解存在沿行方向的前向依赖。将 L 按行块划分后，当前对角块先完成三角求解，再使用已求出的上方右端块更新剩余部分：

```
for each diagonal block i:
    solve L[i:i+bs, i:i+bs] * X[i:i+bs, :] = B[i:i+bs, :]
    B[i+bs:, :] -= L[i+bs:, i:i+bs] * X[i:i+bs, :]
```

其中对角块求解是串行依赖最强的部分，尾部更新是矩阵乘法，适合调用 KBLAS 的 DGEMM。该分解把一个整体的三角求解转化为“对角小块求解加尾部矩阵更新”的重复过程，也为根据矩阵形状选择并行方向提供了接口。三组正式测试规模如表 10 所示，正确性阈值为 10^{-12} 。

用例	矩阵规模	形状特征	最终路径
Case 1	$m = 512, n = 19968$	小 m 、宽 n	递归, leaf = 48, N-split
Case 2	$m = 2432, n = 17024$	中等 m 、宽 n	递归, leaf = 32, N-split
Case 3	$m = 17024, n = 512$	大 m 、窄 n	NB = 224, M-split

表 10: TRSM 三组正式测试规模与路径选择

4.2 基线分析与优化思路

TRSM 的性能不能只由一个固定分块参数决定。三个测试用例的 m 和 n 差异较大：Case 1 与 Case 2 的右端矩阵很宽，尾部更新按 n 分片可以让线程获得足够的连续列工

作；Case 3 的右端矩阵较窄，继续沿 n 切分会使并行度受限，因此改为沿剩余行按 m 分片。最终实现将这一判断固化为按形状分支的 V9 路径，并在此基础上叠加两项改进：宽 n 更新的二维切分（V14）与叶子块求解的 W-technique（V16）。

优化过程中保留了三个原则：第一，矩阵运算的主体交给成熟的 KBLAS DGEMM；第二，KBLAS 在每次调用中固定为单线程，避免其内部线程与外层 OpenMP 嵌套造成过量线程和资源争用；第三，只在足够大的更新上建立 OpenMP 并行区，小矩阵更新直接串行调用 KBLAS，减少线程创建与同步开销。

版本级记录如表 11 所示。不同节点和不同批次的绝对吞吐量仅用于版本演进参考，V16 与同批 V14 的交错对照为 1.051 倍（+5.1%）。

版本	主要路径	综合吞吐量	相对加速
V9	形状感知的递归或块算法	1136.8 GFLOPS	1.000 倍，基线
V14	宽 n 更新二维切分	1154.1 GFLOPS	相对 V9 为 1.015 倍，+1.5%
V16	V14 加 W-technique	1182.1 GFLOPS	相对同批 V14 为 1.051 倍，+5.1%

表 11: TRSM 版本演进的综合加速效果

4.3 优化实现

4.3.1 按形状选择递归或块算法

正式 trsm.c 的入口按矩阵形状分派：当 m 不超过 1024 时使用 leaf 为 48 的递归算法；当 m 大于 1024 且不超过 4096 时使用 leaf 为 32 的递归算法；更大的 m 使用固定块大小 224 的 right-looking 块算法。递归分割点按 32 行对齐，使子问题与 OpenMP 分片保持规则的行边界。

```
if m <= 1024:
    recursive TRSM, leaf = 48
else if m <= 4096:
    recursive TRSM, leaf = 32
else:
    blocked TRSM, NB = 224
```

递归路径先求解左上子问题，再调用 DGEMM 更新右下区域，最后求解右下子问题；大规模块路径沿对角块从上到下推进，在每个块之后更新剩余行。两条路径保持同一个数学递推，只改变任务组织方式，不改变结果语义。

4.3.2 对角块求解与 W-technique (V16)

原实现的 W-technique 采用标量前代消元：行 i 依赖前 $i - 1$ 行的解，形成严格的串行链，聚合速率只有约 110 GFLOPS，在 Case 2 中占据 12.1 ms（占总时间 15%）。V16 将其替换为选择性求逆：先以 $O(n_b^3/6)$ 的串行成本求出 32 至 48 行对角块的逆矩阵（转置存储，使内层乘减成为可向量化的连续访存流），再用原地 `cblas_dtrmm` 按列分片计算 $B_j := L_{jj}^{-1} B_j$ 。串行链被完全消除，求解速率提升到实测的 480 至 860 tri-GFLOPS（形状相关）。64 行以上的大对角块（Case 3 的 224 行块）因求逆串行成本不划算而保持原标量路径，由宏守卫限制。

该替换的数值代价可忽略：最大误差从 6.66×10^{-16} 变为 2.78×10^{-16} 至 4.44×10^{-16} 量级，仍低于阈值四个数量级。对照实验显示 W-technique 的 `leaf = 64` 变体在 Case 2 中使吞吐量从 1433 降至 1032 GFLOPS（约 -28.0%），因此正式版本采用 `leaf = 32` 或 48；经 DGEMM 加临时缓冲的变体也没有增益，缓冲往返流量抵消了计算收益。

4.3.3 尾部更新与二维切分 (V14)

尾部更新的数学形式为

$$B_2 \leftarrow B_2 - L_{21} B_1, \quad (5)$$

实现根据更新规模在两种并行方向之间选择：当 n 大于 4096 时沿列方向分片，每个线程处理连续的 n 列；否则沿行方向分片，每个线程处理连续的剩余行。每个线程调用单线程 KBLAS DGEMM，写入互不重叠的 B_2 区域，既复用 KBLAS 的高效内核，又避免多线程同时写同一输出元素。

纯列分片下 38 个线程都要完整读取一遍乘数面板 L_{21} ，重复读取达到 38 份。V14 将宽 n 更新改为二维切分：38 个线程划分为 2 个行组乘 19 个列组，行组内的 19 个线程共享同一份行区间的 L_{21} ，面板重复读取降到 19 份；代价是 B 面板的重复读取翻倍，但每个线程获得的列条带宽度也翻倍，流式访问效率更高。节点探针显示 Case 2 顶层更新在此切分下从约 1684 提升到约 1835 GFLOPS。该路径由 `TRSM_WIDE_2D` 宏控制，仅在 n 大于 4096 且更新行数不低于 256 时启用，更深的递归小更新仍走纯列分片，Case 3 的行分片不受影响。

当 `rows`、`n`、`bs` 三者乘积较小时，代码不建立 OpenMP 并行区而直接调用 KBLAS，用于抑制递归尾部与最后一个小尾块的 `fork/join` 成本，尤其适合 Case 1 的小对角块与 Case 3 的末尾块。

4.3.4 OpenMP 与 KBLAS 的组合

KBLAS 通过构造函数调用 `BlasSetNumThreads(1)` 把内部 DGEMM 固定为单线程，大更新由外层 OpenMP 负责分片，形成表 12 所示的层次。KBLAS 链接的是 LLVM OpenMP 运行时（BiSheng HPCKit 的 `libomp`），bench 使用 GCC `libgomp`，两个运行

时不做嵌套：对角块的 libgomp 并行区总是在发出多线程 KBLAS 调用之前关闭，避免运行时冲突。

层次	组件	作用
线程配置	OMP_NUM_THREADS=38	使用一个 NUMA 节点上的固定线程数
任务分片	外层 OpenMP	按列或按行切分独立的 DGEMM 子任务
数值内核	单线程 KBLAS	执行每个子矩阵乘，避免嵌套并行
数据放置	numactl -N 1	限制内存和计算的 NUMA 范围

表 12: TRSM 最终版本的并行层次

线程组织的取舍有对照结果：Case 3 尝试将求解与更新流水线重叠时，最佳情况仍下降约 2%，原因是求解线程组导致更新线程组饥饿。因此正式版本保持外层 OpenMP 分片、内层单线程 KBLAS 的层次，不额外引入流水线并发。

4.4 正确性与性能结果

V16 正式结果记录（节点 cn22994, test_runs=5）如表 13 所示，摘要如表 14 所示。三组用例全部通过 10^{-12} 正确性阈值，最大误差为 1.83×10^{-15} ；显示时间总和 215.13 ms，按 bench 的 FLOPs 计量三用例合计约 254.311 GFLOP，综合吞吐量 1182.1 GFLOPS。

用例	规模 $m \times n$	平均时间 / ms	GFLOPS	最大绝对误差	校验
Case 1	512×19968	6.02	869.7710	2.78×10^{-16}	PASS
Case 2	2432×17024	71.08	1416.5514	4.44×10^{-16}	PASS
Case 3	17024×512	138.03	1075.0646	1.83×10^{-15}	PASS

表 13: TRSM V16 正式结果记录

指标	结果	结论
正确性	3/3 PASS, 最大误差 1.83×10^{-15}	满足 10^{-12} 阈值
平均时间总和	215.13 ms	三组正式用例合计耗时
综合吞吐量	1182.1 GFLOPS	按 bench 的 m^2n 计量

表 14: TRSM 正式结果摘要

4.5 尝试、取舍与局限

SME 候选路径的取舍。节点具有 SVE2、SME 与 smef64f64 特征，实测 SVE 向量长度为 512 bit；但 HWCAP2_SVEF64MM 为 0，FMMLA 不能作为可靠的通用路径。KBLAS 的 DGEMM 反汇编包含 SME 的 FMOPA 指令，说明现有库已能使用 SME F64F64 路径。自有 16×32 SME 更新内核在真实 load/store 条件下约为 1737 GFLOPS，加入共享 B 打包与每线程 A 打包后约为 1525 GFLOPS；分段计时修正后的 KBLAS 更新段真实速率约为 1500 GFLOPS，自有内核相对该速率没有实质优势，还需额外的打包、尾块与线程协作逻辑，最终没有接入正式提交。这是基于端到端风险与实测结果的取舍，提交版本优先保留可复现、可验证的 KBLAS 路径。

结构变体的系统否决。V16 之前还系统测过多个结构性变体，全部通过正确性检查但在同批交错对照中负收益：k 聚合（Case 3 慢 2.5%）、扁平分块替代递归（宽 n 用例慢 35%–40%）、持久区域加固定列条带（慢 60%）、宽 n 强制 M-split（慢 66%）、对角求解双行 ILP（Case 3 慢 7.0%）、solve/update 流水线重叠（最好情形慢 2%）、W-technique 经 DGEMM 加临时缓冲（缓冲流量吃掉收益）、W-technique 下 leaf=64（Case 2 慢 28.0%）、扁平 NB=608 重构（无增益）、left-looking 更新族（1399–1672 GFLOPS，全面劣于右视路径的 1884）。这些负结果与 V14、V16 的正收益一起，说明当前算法结构已处于该引擎下的较优位置。

上限分析同样有据可查：节点无 L3（L1d 32 KB + L2 768 KB 私有），单 NUMA 域 DRAM 带宽约 137 GB/s；KBLAS 更新 GEMM 在 Case 2 相关形状族上的历史最佳速率为 1884 GFLOPS，按此上限推算 Case 2 端到端天花板约 1841 GFLOPS，Case 3 的更新段单独就超出相应预算。因此 1800/2000/3000 GFLOPS 的更高目标在该 KBLAS 引擎下数学上不可达，继续逼近需要引入非 KBLAS 的更新内核，而这已被端到端实测证明收益有限。

5 总结与反思

5.1 三题最终结果汇总

三题的最终成绩如表 15 所示：全部用例通过各自的官方误差阈值，性能相对基线均有数量级提升。

5.2 共性经验与反思

三项工作的具体技术各不相同，但可以提炼出几条跨题成立的经验。

第一，绑定与数据放置先于计算优化。ZGEMM 约 35 倍的差距全部来自线程绑定，约 2 倍来自页面放置；CONV 与 TRSM 同样受益于单 NUMA 节点的固定线程数与静态调度。在 NUMA 集群上，绑核命令必须与实际 CPU 配给方式匹配，别人环境里正确的 `numactl` 在自己的环境可能是毒药。

题目	计算结构	最终性能 / GFLOPS	正确性	相对基线
CONV	二维卷积	454.98–534.14（单用例）	4/4 PASS，误差为 0	13.4–14.0 倍
ZGEMM	复数 GEMM	3125.28（三用例合计）	3/3 PASS，最大 4.27×10^{-12}	约 2.8 倍 (FCMLA 移植版)
TRSM	三角求解	1182.1（综合）	3/3 PASS，最大 1.83×10^{-15}	版本演进 +4.0%–5.1%

表 15: 三题最终结果汇总

第二，数值一致性是硬约束，也是可工程化的。CONV 通过保持累加顺序与融合乘法语义做到误差恒为零；ZGEMM 与 TRSM 的算法级改动（分块、求逆）带来的误差增长都在一个数量级以内，仍低于阈值四个数量级。任何改变运算次序的优化都应先给出误差证据。

第三，负结果与正收益同样有价值。三题都积累了大量通过正确性检查但被否决的变体（CONV 的八行版本、ZGEMM 的 Strassen 两级实现与多节点自适应、TRSM 的十余个结构变体），它们划清了当前算法结构在对应硬件上的边界，避免在错误方向上继续投入。

第四，本地模拟对跨 NUMA 集群没有预测力。ZGEMM 的 x86 模拟预测 Strassen 为 1.24 倍加速，集群实测为 0.13 倍；参数甜点在不同节点间同样有 20% 以上的方差。一切结论以同批交错对照为准，是三项工作共同遵守的纪律。

第五，把性能拆解到分段。TRSM 对求解段与更新段分别计时后定位到占 15% 时间的串行链，ZGEMM 用微基准测量发射率上限后才知道常规 GEMM 已到天花板。端到端数字之外的分段证据，是判断“还能不能再快”的唯一可靠依据。

本报告的局限在于：三题的最终成绩都在各自硬件与数学库引擎的可达上限附近，但距离理想峰值仍有差距；三题的测评环境取自 hpc 课程集群或深圳超算，而非官方测评环境。后续工作可以围绕更细粒度的性能建模与判题环境的鲁棒性验证展开。

参考文献

- [1] Hpc101 final project 官方说明. <https://hpc101.zjusct.io/lab/Final-Project/>. Accessed: 2026-09-05.